ARQUITECTURA + FASE 1 CERRADA Y VERIFICADA · 5 SEP 2026

Sistema Central de Punto de Venta e Inventario

El sistema que controla el inventario, el mostrador y la administración de la distribuidora, y que expone una API para conectar la página web existente como un canal más. Una sola fuente de verdad para todas las existencias.

Alcance · POS, inventario, compras, caja, reportes, API

Fuera de alcance · la página web

Entrega · 12 fases · ≈27 semanas · +3 con CFDI

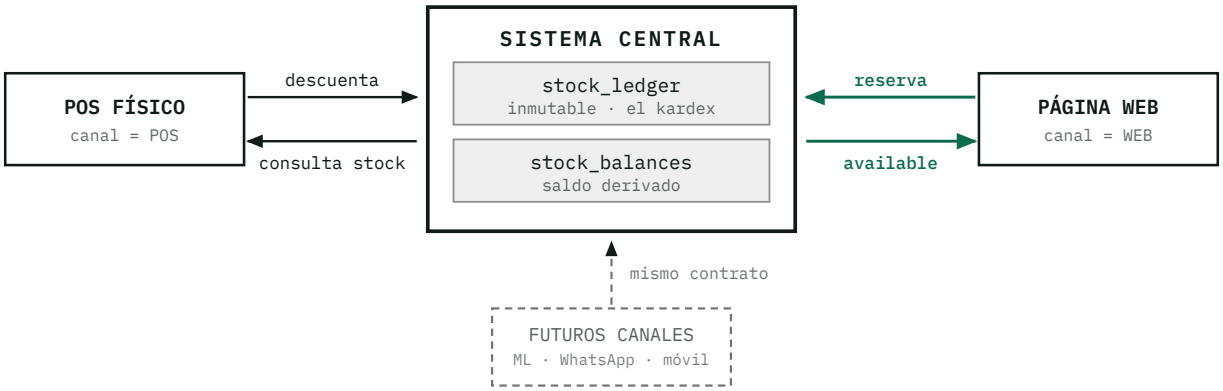
Avance · Fase 1 de 12 cerrada · 155 pruebas en verde

FASE 1 ✓

CERRADA · 155 PRUEBAS

§00 Resumen

Este documento sustituye a Eleventa por un sistema propio construido sobre una idea central: **el inventario no es un número que se edita, es un libro de asientos que se lee**. Todo lo demás — el POS, las compras, la caja, la API de la web — se apoya en esa decisión.



La página web no tiene inventario propio. Consulta `available` por API y reserva antes de cobrar; el POS descuenta contra las mismas dos tablas. Cualquier canal futuro entra por el mismo contrato, sin tocar el motor.

Los tres principios no negociables

01

El stock se mueve, no se edita

Toda variación nace de un asiento inmutable con cantidad anterior, resultante, motivo, usuario y documento. El Kardex no es un reporte: es la tabla misma.

02

Un solo escritor

Una única función escribe existencias, siempre en transacción y con bloqueo de fila. Ningún módulo toca los saldos directamente. Sin esto, la venta doble es cuestión de tiempo.

03

Nada se borra

Se archiva. Precios y costos se fotografían en cada documento: una venta de hace dos años sigue mostrando el costo real de ese día.

Las cuatro decisiones que bloqueaban el arranque

#	DECISIÓN TOMADA	CONSECUENCIA EN EL DISEÑO
D1	**Nube (cloud-first)**	Ninguna computadora del local es punto único de falla. La API queda accesible siempre para la web, sin IP fija ni túneles. Contingencia offline: caché de lectura en Fase 4, venta offline analizada en Fase 12
D2	**CFDI preparado, no improvisado**	Datos fiscales desde la Fase 2 (≈3 días); timbrado real en la Fase 13. **El cajero nunca espera al SAT** — §11
D3	**1 sucursal hoy, multi por diseño**	Modelo multi-almacén desde la Fase 1; la interfaz oculta esa dimensión hasta que exista la segunda ubicación
P6	**Devoluciones configurables**	30 días, reembolso por método original, crédito a favor, cambios. Todo editable desde administración, con auditoría — §12

Separación de responsabilidades

```
VENTA FÍSICA  ──►  POS
VENTA ONLINE  ──►  API
INVENTARIO    ──►  SISTEMA CENTRAL   ← única fuente de verdad para ambos
```

FRONTERA	CÓMO SE GARANTIZA POR CONSTRUCCIÓN
El POS no es dueño del inventario	Solo llama a la API; no tiene lógica de stock propia
La web no es dueña del inventario	Solo consume `/public/v1/**`; recibe `available`, nunca el físico
Nadie escribe stock salvo el centro	`StockService.apply()` es el único escritor, y el rol de base de datos no tiene `UPDATE` sobre el libro de movimientos
Toda venta sabe de dónde vino	`channel_id` obligatorio en cada venta y en cada asiento de inventario
La API pública no puede filtrar costos	Superficie separada físicamente, con DTOs propios y pruebas de contrato en CI

La página web existente no se reconstruye ni se modifica. Solo se desarrolla la API robusta para conectarla después, más la guía de integración, el SDK y el ambiente de pruebas de la Fase 11.

§01 Arquitectura y stack

No elegí por popularidad. El orden de prioridad fue: corrección transaccional, mantenibilidad por un equipo pequeño, rendimiento en el mostrador, facilidad de integración por API, y que el proyecto no dependa de una sola persona.

CAPA	ELECCIÓN	RAZÓN DETERMINANTE
Backend	`NestJS 11 · Node 24 · TS`	Modular por diseño: un módulo por área del negocio. Guards para permisos, interceptores para auditoría, OpenAPI generado del código.
Base de datos	`PostgreSQL 17`	`SELECT FOR UPDATE`, `CHECK`, `NUMERIC` exacto, trigram para búsqueda difusa, particionado del kardex. No es sustituible aquí.
ORM	`Prisma 6 + SQL crudo`	Migraciones versionadas y tipos generados; SQL explícito donde el bloqueo y el rendimiento importan.
Frontend	`React 19 · Vite · TanStack`	Un solo lenguaje en todo el stack y tipos compartidos con el backend.
Búsqueda	`Postgres + índice local`	El POS filtra sin red mientras se teclea; el servidor sigue siendo la verdad al cobrar.
Colas	`Redis 7 + BullMQ`	Webhooks con reintentos, reportes pesados, importaciones.
Dinero	`NUMERIC(14,4)`	Jamás punto flotante. Un centavo de redondeo se acumula durante años.
Identificadores	`UUIDv7 + folios`	Seguros de exponer, ordenables por tiempo, generables sin conexión.

ALTERNATIVAS DESCARTADAS

- **Laravel** — el núcleo de este sistema no es CRUD, es un motor transaccional con eventos y colas; además se pierde el tipado compartido con el frontend, que es el mayor ahorro de mantenimiento aquí.
- **Django** — su admin autogenerado no sirve ni para el POS ni para el panel que necesitas. Se paga el costo de dos lenguajes sin recibir el beneficio.
- **Go / .NET** — rendimiento que no necesitas: el cuello de botella será Postgres, no el runtime. Mayor costo de mantenimiento y de contratación.

- **MongoDB** — descartado de plano. Un sistema de inventario y dinero necesita transacciones y restricciones declarativas como característica central.

Las capas y la regla que las gobierna

DEPENDENCIAS EN UNA SOLA DIRECCIÓN

```
PRESENTACIÓN    Controllers · Guards de permiso · DTOs · Serializadores
                Rate limiting · CORS · Validación · Versionado /api/v1
      ↓
APLICACIÓN      Casos de uso: orquestan, no calculan
                RegistrarVenta · RecibirCompra · AbrirCaja · ReservarStock
                Responsable de: transacción, idempotencia, emisión de eventos
      ↓
DOMINIO         Reglas puras, sin entrada/salida
                Totales · Impuestos · Descuentos · Costeo · Máquinas de estado
      ↓
INFRAESTRUCTURA Prisma · Repositorios · Redis · S3 · Impresora
      ⇕
NÚCLEO          StockService  ← ÚNICO escritor de existencias
TRANSVERSAL     OutboxService · AuditService · IdempotencyService
                SequenceService (folios) · PermissionService
```

REGLA ARQUITECTÓNICA PRINCIPAL

Ningún módulo escribe en `stock_balances` ni inserta en `stock_ledger` directamente. Todo pasa por `StockService.apply()`, que abre transacción, bloquea filas en orden determinista, valida, inserta el asiento, actualiza el saldo y encola el evento. Todo, o nada.

Se hace cumplir de tres formas independientes: revisión de código, una regla de ESLint que prohíbe importar el repositorio de stock fuera del módulo de inventario, y **permisos de PostgreSQL** — el rol de aplicación no tiene `UPDATE` ni `DELETE` sobre el libro de movimientos.

Estructura del repositorio

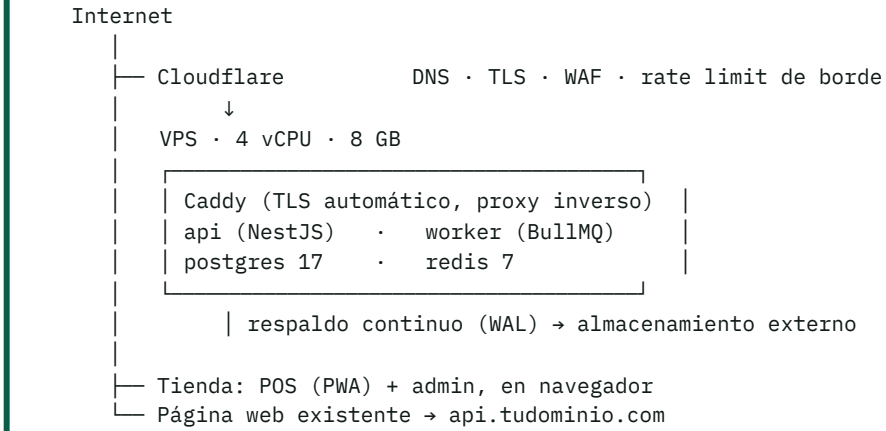
MONOREPO · PNPM WORKSPACES + TURBOREPO

```
apps/
  api/                  NestJS — el Sistema Central
    src/core/           transacciones · auditoría · idempotencia · outbox · folios
    src/modules/        auth · org · catalog · inventory ★ · pricing · sales
                        payments · cash · purchases · customers · reports
                        imports · webhooks · channels · settings
    src/public-api/v1/  ← superficie para canales externos, versionada y estable
    src/jobs/           workers BullMQ
  admin/                React — panel administrativo (responsive real)
  pos/                  React PWA — punto de venta (primero teclado)
  print-bridge/         servicio local: ESC/POS + cajón de dinero
packages/
  shared-types/         tipos compartidos API ⇔ frontends
  domain-rules/         cálculos puros usados en ambos lados
infra/                  docker-compose · pgBackRest · verificación de restauración
docs/                   esta documentación
```

Por qué `public-api/v1` está separado: la API que consume la página web es un contrato con garantía de estabilidad. Separarla físicamente evita que un refactor interno rompa la tienda online sin que nadie se entere, y hace imposible por construcción que un costo se filtre hacia un cliente externo.

Dónde corre — cloud-first (decidido)

VPS + CLOUDFLARE · ≈25–40 USD AL MES



- **Ninguna computadora del local es punto único de falla.** Si se quema la PC del mostrador, se abre el POS en otra máquina o en una tablet y se sigue vendiendo: no hay nada irreemplazable en el local.

- La API es pública desde el día uno: la integración con la web no depende de la infraestructura de la tienda.
- Respaldo automático fuera del local — protege contra robo, incendio y disco muerto.
- Una sola versión desplegada, no N instalaciones que divergen.

El único riesgo asumido es la caída del internet de la tienda, y se mitiga en dos tiempos: desde la Fase 4 el POS cachea catálogo, precios y último stock conocido; en la Fase 12 se analiza la venta offline con datos reales de tu conexión, siempre con la consistencia del inventario como prioridad (§07).

Si la conexión resultara crónicamente inestable, la arquitectura admite añadir una réplica de solo lectura en la tienda manteniendo todas las escrituras contra la nube. Es una adición, no un rediseño.

NOTA DEL ENTORNO

Esta máquina tiene Node 24 y Git, pero **no tiene Docker ni PostgreSQL**. La Fase 1 arranca instalando Docker Desktop — levanta Postgres y Redis con un solo comando — o bien PostgreSQL 17 y Redis nativos para Windows.

§02 Modelo de datos

52 tablas. El esquema completo se crea en la Fase 1 aunque muchas tablas se usen después: diseñar las relaciones de una vez evita migraciones que tengan que reescribir historial de inventario más adelante.

GRUPO	TABLAS	PIEZAS PRINCIPALES
Organización	4	companies · branches · warehouses · terminals
Seguridad	8	users · roles · permissions · sessions · api_clients · api_keys
Catálogo	12	products · product_variants · barcodes · options · brands · categories
Inventario ★	7	stock_ledger · stock_balances · reservations · counts · transfers
Precios	6	price_lists · price_list_items · promotions · promotion_rules
Ventas	7	sales · sale_items · returns · return_items · held_sales
Caja y pagos	4	cash_sessions · cash_movements · payments · payment_methods
Compras	7	purchase_orders · receipts · supplier_prices · suppliers
Clientes	3	customers · customer_account_entries · credit_notes
Canales	4	channels · external_orders · integrations · webhook_endpoints
Plataforma	10	outbox_events · idempotency_keys · audit_logs · sequences · imports

Convenciones aplicadas a todo

ASPECTO	REGLA
Llave primaria	UUID v7 — ordenable por tiempo, seguro de exponer, generable sin conexión
Folio visible	Columna aparte, por secuencia de sucursal y tipo de documento: `A-001-000123`
Dinero	`NUMERIC(14,4)`. Jamás `FLOAT`
Fechas	`TIMESTAMPTZ` en UTC, presentadas en `America/Mexico_City`
Borrado	`deleted_at`. Nunca `DELETE` en tablas con historial
Edición concurrente	`version` + `If-Match` en productos y precios

La decisión que evita la mitad de los bugs de variantes

Un producto sin variantes visibles **sí tiene una variante interna**. El inventario, las ventas y las compras apuntan **siempre** a `product_variant_id`, nunca a `product_id`. Esto elimina el `if (producto.tieneVariantes)` repartido por todo el código, que es la fuente número uno de errores en los POS con variantes. La interfaz oculta la variante cuando solo hay una; el motor nunca la ve como caso especial.

FAJA POSTQUIRÚRGICA MODELO X — UNA ALTA, OCHO VARIANTES

```
products
  └─ product_options        Color            Talla
       └─ option_values     Negro · Beige    S · M · L · XL

product_variants   (8 filas, cada una con SKU, código de barras y stock propios)
  FJ-0012-NEG-S   Negro / S      FJ-0012-BGE-S   Beige / S
  FJ-0012-NEG-M   Negro / M      FJ-0012-BGE-M   Beige / M
  FJ-0012-NEG-L   Negro / L      FJ-0012-BGE-L   Beige / L
  FJ-0012-NEG-XL  Negro / XL     FJ-0012-BGE-XL  Beige / XL

barcodes           1..N por variante  ← el del fabricante Y el interno que imprimes tú
                   UNIQUE(code) global entre los no archivados
```

Los códigos de barras son tabla aparte por una razón práctica: los proveedores reutilizan y cambian códigos EAN. Con una sola columna en el producto habría que elegir cuál conservar; como tabla, ambos escanean bien. Si al dar de alta hay colisión, el sistema muestra a qué variante pertenece en vez de un error seco.

Las dos tablas que sostienen todo

```
STOCK_LEDGER · INMUTABLE · PARTICIONADA POR MES

occurred_at · variant_id · warehouse_id · condition (GOOD|DAMAGED|QUARANTINE)
movement_type   PURCHASE_RECEIPT · SALE · RETURN_IN · ADJUSTMENT_IN/OUT
                COUNT_CORRECTION · TRANSFER_OUT/IN · DAMAGE · INITIAL_LOAD
direction · qty
qty_before      ← cantidad anterior          qty_after   ← cantidad resultante
unit_cost       ← costo de ESTE movimiento   avg_cost_after
reason_code · notes
reference_type · reference_id · reference_folio     ← el documento que lo originó
channel_id · user_id · terminal_id · request_id     ← quién, desde dónde, en qué canal

GRANT INSERT, SELECT — sin UPDATE, sin DELETE. Corregir = asiento contrario.
```

```
STOCK_BALANCES · DERIVADO · RECONSTRUIBLE AL 100 %

PK (variant_id, warehouse_id, condition)

qty_on_hand · qty_reserved · avg_cost · last_movement_at
qty_available     ← COLUMNA GENERADA: qty_on_hand − qty_reserved
                    no se almacena aparte; un valor duplicado se desincroniza

CHECK (qty_on_hand  >= 0)
CHECK (qty_reserved >= 0)
CHECK (qty_reserved <= qty_on_hand)
```

POR QUÉ IMPORTAN ESOS TRES CHECK

Son la última línea de defensa. Si un bug futuro intentara sobrevender, PostgreSQL aborta la transacción. **Prefiero un error 500 ruidoso a inventario silenciosamente corrupto.** Un job nocturno recalcula todos los saldos desde el libro de movimientos y reporta discrepancias; en operación normal debe reportar cero siempre.

Dos correcciones al modelo que pediste

Ambas son el mismo error aplicado a dos dominios distintos, y las dos se corrigen con la misma medicina que el inventario. Detalle en §10 · P1 y P2.

- **Precios duplicados** — pediste campos `precio_mayoreo` y `precio_distribuidor` en el producto *y además* listas de precios. Son dos modelos del mismo dato y acabarán contradiciéndose. Propuesta: una sola fuente (`price_lists`), con la misma pantalla de siempre por encima.

- **Saldo del cliente como campo editable** — un número que nadie puede explicar cuando el cliente reclama. Propuesta: cuenta corriente por asientos; el saldo es la suma, y hay estado de cuenta imprimible.

§03 Inventario

En Eleventa el stock es un número en una columna: cuando está mal, no hay forma de saber por qué. Aquí el diseño es el opuesto — ante cualquier duda, el Kardex responde con nombre, hora, documento y motivo.

El Kardex es la tabla, no un reporte

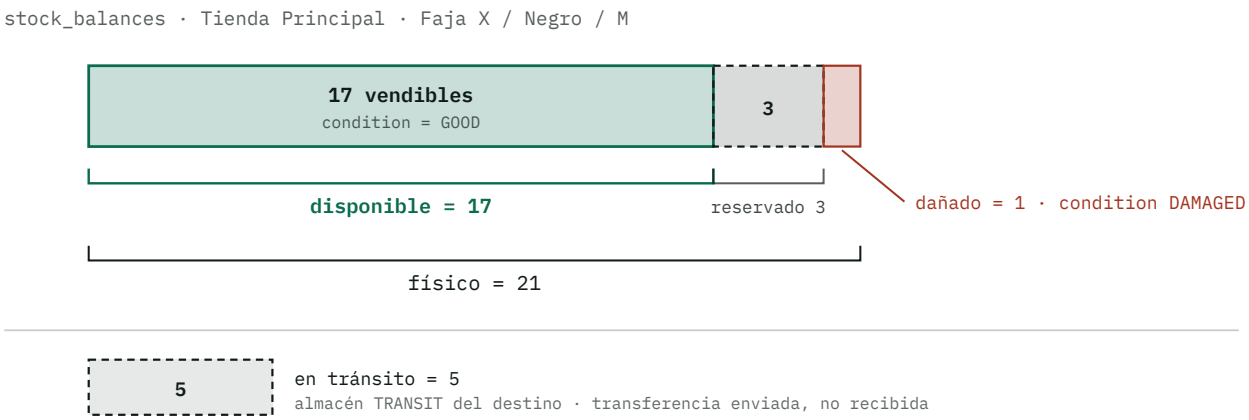
FAJA POSTQUIRÚRGICA MODELO X — NEGRO / M · FJ-0012-NEG-M · TIENDA PRINCIPAL

Fecha	Hora	Movimiento	Referencia	Entrada	Salida	Stock	Costo	Usuario
05/09/2026	09:12	Carga inicial	INIT-0001	20.0	-	20.0	400.00	Sistema
05/09/2026	11:40	Venta	A-001-000045	-	1.0	19.0	400.00	María
05/09/2026	13:05	Compra recibida	REC-000012	30.0	-	49.0	450.00	Jesús
		↳ el costo promedio pasa de 400.00 a 430.61						
05/09/2026	16:22	Venta web	W-000003	-	2.0	47.0	430.61	API/WEB
06/09/2026	10:15	Devolución	DEV-000002	1.0	-	48.0	400.00	María
		↳ reingresa al costo con el que salió, no al promedio actual						
06/09/2026	18:00	Ajuste (merma)	AJU-000004	-	1.0	47.0	430.61	Jesús
		↳ Motivo: producto dañado en bodega						

Saldo actual: 47.0 Valor: $20,238.67

Cada renglón es clicable y lleva al documento que lo originó. Filtros por rango de fechas, almacén, tipo de movimiento, usuario y canal. Exportable a Excel, CSV y PDF.

De dónde sale cada tipo de stock



Los siete tipos de stock que pediste no son siete columnas: se derivan de dos tablas, por lo que no pueden contradecirse entre sí. La API pública responde **siempre** `available` (17), nunca el físico (21) — es la decisión que evita que la web venda lo que ya está apartado.

TU CONCEPTO	CÓMO SE CALCULA
Físico	`SUM(qty_on_hand) en el almacén, todas las condiciones`
Disponible	`qty_on_hand − qty_reserved · condition = GOOD`
Reservado	`reservas en estado HELD (con vencimiento)`
Comprometido	`reservas en estado COMMITTED (sin vencimiento)`
En tránsito	`qty_on_hand en el almacén type = TRANSIT del destino`
Dañado	`qty_on_hand con condition = DAMAGED`
Apartado	`reservas COMMITTED con purpose = LAYAWAY`

Reservado y comprometido no son dos mecanismos, son dos estados del mismo. Implementarlos como contadores independientes duplicaría la lógica de liberación y garantizaría que se desincronicen (§10 · P3).

Costeo: promedio ponderado móvil

```
Estado:    10 piezas · promedio $400   →   valor $4,000
Entra:     30 piezas ·        $450     →   valor $13,500
                                           ————————————
Nuevo:     40 piezas                       valor $17,500
Promedio:  $17,500 / 40 = $437.50
```

Elegí promedio y no PEPS porque PEPS obliga a mantener capas de costo por lote y consumirlas en orden, lo que multiplica la complejidad de cada venta y cada devolución. Para fajas — producto no perecedero, sin caducidad, con costos que varían de forma gradual — el promedio da un margen prácticamente idéntico con una fracción del riesgo de bugs. El esquema deja la puerta abierta: el libro de movimientos ya guarda el costo unitario de cada asiento, que es exactamente lo que PEPS necesitaría.

TU REQUISITO 35, RESUELTO EN DOS CAPAS

Que una venta hecha cuando el costo era $400 no cambie al subir el costo a $470 se garantiza estructuralmente, no por convención:

- `sale_items.unit_cost` se **copia** en el momento de la venta. Es un dato de la venta, no una referencia al producto. Nada lo puede alterar.
- `stock_ledger.avg_cost_after` congela el costo de cada movimiento, y `supplier_prices` conserva el costo negociado con cada proveedor a lo largo del tiempo.

Búsqueda: el punto donde superas a Eleventa

Estrategia de dos niveles. En el **servidor**, cada variante mantiene por trigger un texto normalizado — sin acentos, en minúsculas — que concatena nombre, SKU, código de barras, marca, categoría, color, talla, código de proveedor y sinónimos, con índices GIN de trigram y búsqueda de texto completo.

DICCIONARIO DE SINÓNIMOS — TABLA EDITABLE, NO CÓDIGO

```
postquirurgica  →  post · postqx · pqx · cirugia
reductora       →  reduct · faja reductora
levanta gluteos →  levantacola · push up · levanta pompa
negro           →  negra · neg · black
beige           →  bge · piel · nude · cafe claro
chica / mediana →  s · ch · small   |   m · med · medium
grande / XG     →  l · g · large    |   xl · eg · xg
```

ESCRIBES	ENCUENTRA	MECANISMO
`post m`	Faja postquirúrgica, talla M	sinónimos + multi-token AND
`negra m`	Variantes negras en mediana	valores de opción indexados
`FJ-00`	Coincidencias por SKU parcial	prefijo, puntuación alta
`postquirurjica`	Postquirúrgica (con el error)	trigram · tolerancia a typos
`7501234567890`	La variante exacta	código de barras, sin clics

En el **POS**, al abrir turno se descarga el catálogo comprimido —unos 2 MB para 20 000 variantes — a IndexedDB y se construye un índice en memoria. El resultado aparece **mientras se teclea, sin red**. Al agregar la línea al ticket, el servidor confirma existencia y precio reales. Ese es el diseño que hace que el mostrador se sienta más rápido: **no espera al servidor para mostrar, pero nunca confía en el caché para cobrar**.

El lector de código de barras se comporta como teclado. El POS lo distingue de un humano por la velocidad entre pulsaciones (más de 50 caracteres por segundo) y agrega el artículo **sin que el cajero tenga que hacer clic en el campo de búsqueda primero**. Es un detalle pequeño que cambia la velocidad real del mostrador.

§04 Flujos operativos

Cada flujo distingue explícitamente lo que ocurre **dentro de la transacción** —atómico, todo o nada— de lo que ocurre **después del commit**, asíncrono y con reintentos.

Venta en el mostrador

```
POST /SALES · IDEMPOTENCY-KEY OBLIGATORIA

╔═ TRANSACCIÓN ═══════════════════════════════════════════════════╗
║  a. Verifica clave de idempotencia → si existe, devuelve lo guardado ║
║  b. Valida sesión de caja abierta en esta terminal                   ║
║  c. Bloquea saldos: SELECT … FOR UPDATE en orden determinista        ║
║  d. RECALCULA precios en el servidor  ← nunca confía en el cliente   ║
║  e. Aplica promociones · impuestos · descuentos · redondeo           ║
║  f. Valida SUM(pagos) >= total                                       ║
║  g. Genera folio por secuencia de sucursal                           ║
║  h. INSERT sales + sale_items                                        ║
║       ↳ copia SKU, nombre, PRECIO y COSTO — fotografía inmutable     ║
║  i. INSERT payments  (pago mixto = N filas, sale gratis)             ║
║  j. StockService.apply(salidas) → asiento + saldo                    ║
║  k. Si consume reservas → CONSUMED y libera qty_reserved             ║
║  l. INSERT cash_movements (solo la parte en efectivo)                ║
║  m. Auditoría + Outbox: SALE_CREATED · SALE_COMPLETED · STOCK_CHANGED ║
╚═══════════════════════ COMMIT ═══════════════════════════════════╝

DESPUÉS DEL COMMIT
  · Imprime el ticket y abre el cajón     · Webhooks hacia la web
  · Actualiza métricas del cliente        · Evalúa alertas de bajo stock
```

El precio se recalcula siempre en el servidor. El POS muestra un estimado; el servidor calcula el definitivo con el mismo servicio que usa la web. Si el precio cambió mientras el ticket estaba abierto, el sistema avisa antes de cobrar en vez de cobrar mal en silencio.

Suspender una venta no reserva inventario, por decisión deliberada: un ticket olvidado no debe congelar mercancía. Al recuperarlo se revalida todo y se avisa de lo que cambió.

Compra con recepción parcial

La orden de compra **no toca el inventario**; solo la recepción genera asientos. Esa separación resuelve tu ejemplo sin ningún caso especial:

```
Orden PO-000015 · Fajas Colombia S.A.S.
  Faja X / Negro / M      ordenadas 100    recibidas   0    pendientes 100

  ↓ llega el primer embarque: 80 piezas

  RECEPCIÓN REC-000031
    · Prorratea flete y aduana → landed_unit_cost (costo REAL puesto en bodega)
    · StockService.apply(entrada 80) → recalcula el promedio ponderado
    · qty_received = 80 → estado de la orden: PARTIALLY_RECEIVED
    · INSERT supplier_prices (historial de costo del proveedor)

  Resultado    ordenadas 100    recibidas  80    pendientes  20
               Inventario: +80    ← únicamente lo recibido
```

ALGO QUE FALTA EN TUS REQUISITOS

Importas de Colombia, así que el costo real de una faja no es el de la factura del proveedor: incluye flete internacional, aduana e impuestos de importación. Si el sistema registra solo el costo de factura, **todos tus márgenes serán optimistas de forma sistemática** y las decisiones de precio se tomarán sobre datos falsos. Ya lo incluí en el modelo — los gastos se prorratean por valor o por peso (§10 · P5).

Devolución

Siempre ligada a la venta original, siempre con el precio y el costo originales. El bloqueo de la línea de venta impide que dos terminales devuelvan más de lo vendido.

- **El estado decide el destino:** `GOOD` regresa al almacén de venta; `DAMAGED` entra al almacén de mermas. Sin esto, una faja devuelta usada vuelve a mostrarse como vendible.
- **Tres resoluciones:** reembolso, crédito a favor (nota de crédito) o cambio, que genera una venta nueva y cobra o abona la diferencia.
- **Las reglas son configurables**, no están en el código: días máximos, si se devuelve efectivo de una compra con tarjeta, monto que exige autorización (§10 · P6).

Caja: arqueo con diferencia explicada

```
CORTE Z · SESIÓN CS-000042 · CAJA 1 · MARÍA LÓPEZ

EFECTIVO
  Fondo inicial                                  $  2,000.00
  + Ventas en efectivo              (47 tickets) $ 38,450.00
  + Entradas de efectivo             (1)         $    500.00
  − Devoluciones en efectivo         (2)         $ −1,200.00
  − Retiros a bóveda                 (1)         $−20,000.00
  − Gastos                           (1)         $   −350.00
  ──────────────────────────────────────────────────────────
  EFECTIVO ESPERADO                              $ 19,400.00   ← calculado
  EFECTIVO CONTADO                               $ 19,350.00   ← por denominación
  DIFERENCIA                                     $    −50.00   ⚠ FALTANTE

OTROS MÉTODOS (informativos, no afectan el cajón)
  Tarjeta (31) $52,300.00  ·  Transferencia (4) $8,900.00  ·  Crédito (2) $3,400.00
  ──────────────────────────────────────────────────────────
  VENTA TOTAL DEL TURNO            (84 tickets)  $103,050.00
  Ticket promedio                                $  1,226.79
```

`expected_cash` **nunca** es un campo editable: se calcula sumando los movimientos. El conteo se captura por denominación y el sistema suma —menos errores que teclear un total. **No se puede cerrar una caja con diferencia sin escribir un motivo**, y por encima del umbral configurado se exige PIN de supervisor.

Importación masiva

```
1 SUBIR        CSV o Excel · detección automática de columnas
2 MAPEAR       columna del archivo → campo del sistema (se guarda la plantilla)
3 VALIDAR      fila por fila, SIN escribir nada:
                 ✓ 1,203 filas listas
                 ⚠    45 actualizarán un producto existente (coincidencia por SKU)
                 ✗    12 con error:
                      fila  89 → SKU duplicado dentro del archivo
                      fila 204 → categoría "Fajaz" no existe (¿"Fajas"?)
                      fila 511 → costo no numérico: "450,00 COP"
4 PREVISUALIZAR  tabla con el ANTES y el DESPUÉS de cada cambio
5 CONFIRMAR      exige escribir la palabra de confirmación
6 APLICAR        en lotes, con progreso, y REVERSIBLE
```

La carga de inventario inicial genera asientos `INITIAL_LOAD`, no un `UPDATE` de saldos: el arranque del sistema también queda auditado desde el día uno.

§05 API de integración

RECORDATORIO DE ALCANCE

Aquí **no se desarrolla la página web**. Se define y construye el contrato que la página web existente consumirá como cliente externo. La web se mantiene por separado y no se toca.

Dos superficies separadas físicamente

	API INTERNA	API PÚBLICA
Ruta	`/api/v1/**`	`/public/v1/**`
Consumidores	Panel admin, POS	Página web, futuros canales
Autenticación	JWT de sesión	Llave de API + firma HMAC
Estabilidad	Cambia con el frontend	**Contrato con garantía de versión**
Costos y márgenes	Según permiso	**Nunca expuestos**

Si la web consumiera los mismos endpoints que el panel, cualquier refactor interno rompería la tienda online sin aviso, y un descuido expondría tus costos. La separación física hace que ambas cosas sean imposibles por construcción, no por revisión.

Lo que la web puede pedir

```
LECTURA
  GET  /public/v1/products            paginado · ?updated_since= para sincronizar
  GET  /public/v1/variants            ?filter[in_stock]=true
  GET  /public/v1/categories          árbol completo
  GET  /public/v1/stock               ?variant_ids=…  hasta 200 por llamada
  POST /public/v1/prices/quote        cotiza un carrito COMPLETO con promociones

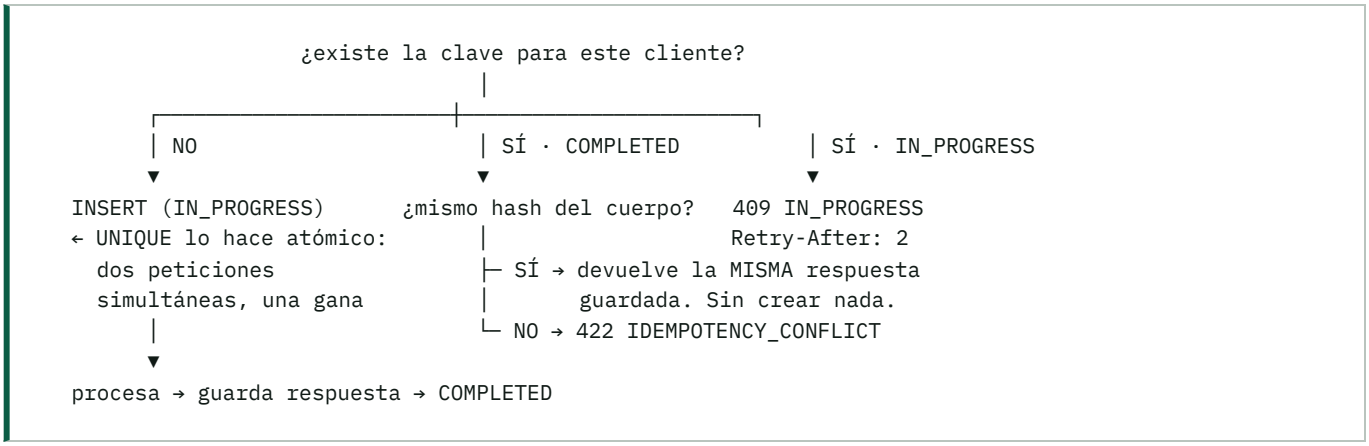
ESCRITURA                             ← todas exigen Idempotency-Key
  POST   /public/v1/reservations      aparta stock antes de cobrar
  DELETE /public/v1/reservations/{id} libera
  POST   /public/v1/orders            confirma el pedido → genera la venta
  POST   /public/v1/orders/{id}/cancel
  POST   /public/v1/orders/{id}/returns
```

DOS REGLAS QUE EVITAN LOS ERRORES CLÁSICOS DE E-COMMERCE

- **La web nunca recibe el stock físico**, solo `available`. Si recibiera el físico, vendería lo que ya está apartado para otro pedido.
- **La web no calcula precios**: los pide con `/prices/quote` y los muestra. Así el mostrador y la web jamás pueden diferir. Si el pedido llega con un precio distinto al calculado, la respuesta es `409 PRICE_MISMATCH` con ambos valores, no una aceptación silenciosa.

Idempotencia — tu requisito 27

Si la web reenvía el mismo pedido por un timeout, **no se registran dos ventas**. La clave se inserta con restricción `UNIQUE` **dentro de la misma transacción** de la venta, así que no hay ventana de carrera.



Triple protección contra el pedido duplicado: la `Idempotency-Key` a nivel de protocolo; `UNIQUE(channel_id, external_id)` a nivel de negocio —aunque la web mande dos claves distintas para el mismo pedido, el identificador externo lo detiene—; y el hecho de que una reserva solo puede consumirse una vez.

Webhooks con garantía de entrega

PATRÓN OUTBOX TRANSACCIONAL

```
TRANSACCIÓN de la venta
├── INSERT sales
├── INSERT stock_ledger
└── INSERT outbox_events        ← el evento NACE con el dato
COMMIT
     │
     ▼  relay: LISTEN/NOTIFY + barrido cada 5 s como respaldo
BullMQ → entrega HTTP firmada (HMAC-SHA256)
     │
     ├── 2xx           → SUCCESS
     └── error/timeout → 1s · 5s · 30s · 2m · 10m · 1h · 6h  (7 intentos)
                         → EXHAUSTED + alerta
                         → tras 20 fallos seguidos el endpoint se desactiva solo
```

- Es **imposible** que exista una venta sin su evento — nacen en la misma transacción.
- Es **imposible** que se dispare un webhook de una venta que hizo rollback.
- Garantía **al-menos-una-vez**: la web debe deduplicar por `X-Event-Id`. Va documentado en la guía de integración porque es la responsabilidad del consumidor.
- Panel con historial de entregas, reintento manual y **reproducción de eventos por rango de fechas** para cuando la web estuvo caída.

EVENTOS PUBLICADOS

```
PRODUCT_CREATED · PRODUCT_UPDATED · PRODUCT_ARCHIVED · VARIANT_PRICE_CHANGED
STOCK_CHANGED   · LOW_STOCK_REACHED · OUT_OF_STOCK
SALE_CREATED    · SALE_COMPLETED · SALE_CANCELLED · RETURN_CREATED
PURCHASE_RECEIVED · CASH_SESSION_CLOSED
ORDER_CREATED   · ORDER_PAID · ORDER_FULFILLED · ORDER_CANCELLED
RESERVATION_CREATED · RESERVATION_EXPIRED · RESERVATION_RELEASED
```

Se entrega además OpenAPI 3.1 generado del código, un SDK en TypeScript, colección de Postman, un ambiente de pruebas con datos ficticios, y pruebas de contrato en CI: **quitar un campo público hace fallar la compilación**, no la tienda.

§06 Seguridad, roles y auditoría

El rol Cajero, exactamente como lo pediste

PUEDE	NO PUEDE
Vender y cobrar	Modificar costos
Imprimir y reimprimir tickets	Ajustar inventario manualmente
Suspender y recuperar ventas	Eliminar o archivar productos
Abrir y cerrar su propia caja	Cambiar precios de lista
Devolver, con autorización	Ver reportes de utilidad
Aplicar descuento hasta su límite	Ver ventas de otros cajeros

LA SEPARACIÓN DE PERMISOS MÁS IMPORTANTE

`product.cost.view` es un permiso **distinto** de `product.view`. El cajero necesita ver el producto; no necesita ver cuánto te costó. La separación se aplica en el **serializador**, no en la interfaz: aunque alguien llamara al API directamente, el campo simplemente no viene en la respuesta.

Autorización de supervisor: cuando un cajero intenta algo que excede su permiso, el sistema no lo bloquea con un error seco — pide el PIN de alguien autorizado, ejecuta la acción y la registra con `authorized_by_user_id`. Es como funciona un mostrador real, y deja rastro.

Auditoría

TU EJEMPLO DEL REQUISITO 20, CON LO QUE HACE FALTA PARA INVESTIGAR DE VERDAD

```
{
  "occurred_at": "2026-09-05T16:42:11-06:00",
  "user_name":   "Juan Pérez",
  "action":      "product.price.update",
  "entity_label":"Faja Postquirúrgica X - Negro / M (FJ-0012-NEG-M)",
  "changes":     { "base_price": { "before": "899.00", "after": "999.00" } },
  "reason":      "Ajuste por incremento de costo de importación",
  "authorized_by_user_id": null,
  "ip_address":  "189.203.44.12",
  "request_id":  "01J8K4...",        ← correlaciona con todos los logs
  "branch_id":   "01J8K1...",
  "channel_id":  "POS"
}
```

INMUTABILIDAD IMPUESTA POR LA BASE DE DATOS, NO POR DISCIPLINA

```
REVOKE UPDATE, DELETE ON audit_logs   FROM app_role;
REVOKE UPDATE, DELETE ON stock_ledger FROM app_role;
GRANT  INSERT, SELECT  ON audit_logs, stock_ledger TO app_role;
```

Un registro de auditoría editable no sirve para auditar nada. Retención de 7 años, particionado mensual. Desde cualquier producto, venta o cliente hay un botón *«Ver historial»* con su línea de tiempo completa.

Protección específica del inventario

Más allá de la seguridad genérica, aquí el activo a proteger es la mercancía. Estos controles no existen en Eleventa y son, en la práctica, el mayor retorno de tener sistema propio: **el inventario deja de ser una caja negra**.

RIESGO	CONTROL
Ajuste de inventario para tapar un robo	Permiso restringido · motivo tipificado obligatorio · reporte semanal de ajustes por usuario
Venta a precio cero o descuento abusivo	Descuento ilimitado es permiso aparte · por encima del umbral exige PIN y queda registrado
Cancelar tickets para sacar efectivo	Permiso restringido · motivo obligatorio · aparece en el corte Z
Devolución falsa	Exige la venta original · reporte de devoluciones por cajero
Faltantes de caja recurrentes	Historial de diferencias por cajero visible en el panel
Fuga del catálogo con costos	Permiso de costo separado · toda exportación queda auditada

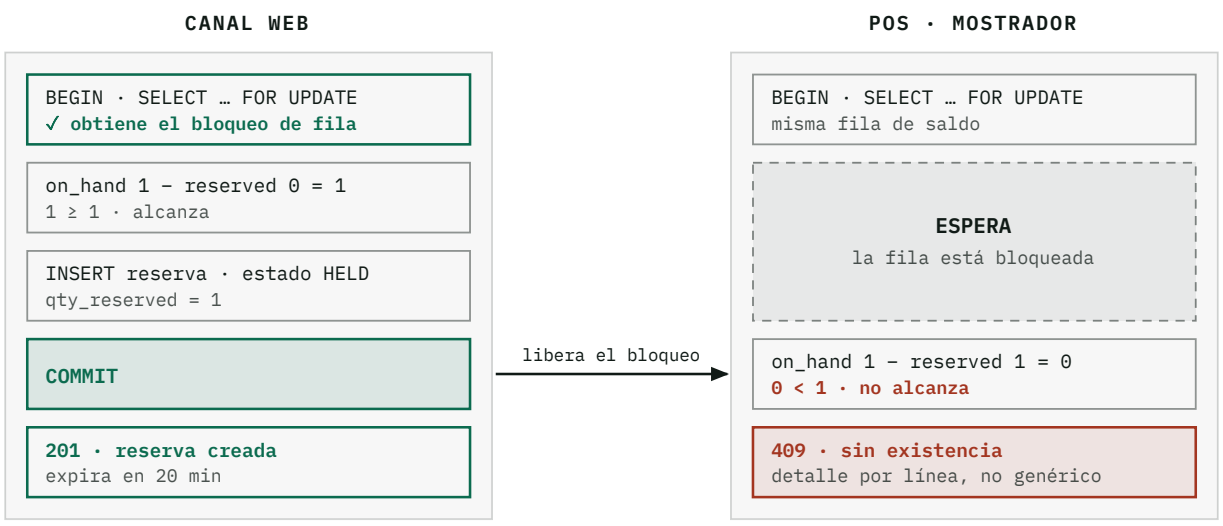
ENDURECIMIENTO GENERAL

- **argon2id** para contraseñas y PIN (no bcrypt: mejor resistencia a GPU). MFA obligatorio para Administrador.
- **Refresh rotatorio con detección de reutilización:** si llega un token ya consumido —la señal clásica de robo— se revocan todas las sesiones del usuario y se registra.
- **Validación estricta:** cualquier campo no declarado en el DTO hace fallar la petición. Cierra la asignación masiva.
- **SQL siempre parametrizado.** La concatenación de cadenas en SQL está prohibida por regla de ESLint y se verifica en CI.
- **Nunca se almacenan datos de tarjeta.** Solo últimos 4 dígitos y número de autorización, como referencia. Es una decisión deliberada que deja el sistema fuera del alcance pesado de PCI.
- **Errores hacia afuera:** código estable, mensaje entendible, identificador de petición. Nunca stack trace, nunca SQL, nunca nombres de tabla.

§07 Concurrencia

Stock igual a 1. Al mismo tiempo, el cliente A compra en la web y el cliente B en el mostrador. Esta sección responde tu requisito 24, que es la pregunta técnica más importante de todo el proyecto.

En un sistema ingenuo —leer stock, validar en memoria, escribir stock— ambas peticiones leen `1`, ambas concluyen «sí hay», y ambas escriben `0`. Se vendieron dos, había una, y el cliente que no recibe su faja se entera tres días después.



El bloqueo de fila serializa a los dos compradores. El tiempo corre hacia abajo; la espera del POS dura milisegundos, imperceptible para el cajero. Además, las restricciones `CHECK` de la base actúan como última defensa: un bug produce una excepción, nunca inventario corrupto.

El cajero ve de inmediato «este artículo está apartado para un pedido web en curso, disponible 0», y decide con información en vez de descubrir el faltante al día siguiente. Si el pedido web se cae en el pago, la reserva expira en 20 minutos y la pieza vuelve sola al mostrador.

Las cinco capas

#	CAPA	QUÉ GARANTIZA
1	**Transacción**	Todo o nada. Sin estados intermedios visibles
2	**Bloqueo pesimista de fila**	Serializa a quienes compiten por el mismo producto — y solo a ellos
3	**Restricciones CHECK**	Un bug produce error, no corrupción
4	**Reservas**	Aparta el stock antes de cobrar; reduce la ventana a cero
5	**Idempotencia**	Un reintento no crea una segunda venta

POR QUÉ PESIMISTA Y NO OPTIMISTA

Bajo conflicto, el bloqueo pesimista hace que el segundo *espere* unos milisegundos; el optimista hace que *falle* y haya que reintentar, lo que significa bucles de reintento por todo el código y errores esporádicos e inexplicables para el cajero. Los conflictos reales sobre el mismo SKU son raros y las transacciones duran milisegundos: pesimista es más simple y más predecible. El optimista sí se usa, pero para otra cosa —la edición de catálogo y precios—, donde dos personas editando el mismo producto deben enterarse del conflicto en vez de pisarse.

PREVENCIÓN DE BLOQUEOS MUTUOS

El **orden determinista de bloqueo** (siempre ordenado por variante, almacén y condición) elimina la causa clásica del deadlock por sí solo. Se refuerza con `lock_timeout = 3s`, para que una caja nunca se cuelgue indefinidamente, y `statement_timeout = 10s`.

La tensión honesta: offline y un solo inventario

CONTRADICCIÓN REAL EN LOS REQUISITOS

Pediste inventario central como única verdad *y* que evaluara el trabajo sin conexión. Son objetivos que se contradicen parcialmente: si el POS vende sin conexión, no puede saber si la web acabó de vender la última pieza. **Cualquier sistema que prometa ambas cosas al 100 % está escondiendo el conflicto.**

NIVEL	ALCANCE	RIESGO	CUÁNDO
0 · Lectura	Catálogo, precios y último stock conocido en el dispositivo	**Ninguno**	Fase 4
1 · Venta en efectivo	Solo efectivo, sin crédito ni promociones complejas. Cola con UUID local que es su propia clave de idempotencia	**Sobreventa acotada**	Fase 12
2 · Todo offline	—	**Descartado**	No se hará

Mitigación del nivel 1: un colchón de seguridad configurable —las últimas N piezas de cada variante no se venden sin conexión— que en productos de baja rotación, la mayoría de tu catálogo, elimina el riesgo casi por completo. Más una banda visible en el POS, un reporte de conflictos al sincronizar, y confirmación de gerente pasadas 4 horas sin conexión.

Cómo se demuestra que funciona

LA CORRECCIÓN BAJO CONCURRENCIA NO SE AFIRMA, SE PRUEBA

```
it('no permite sobrevender con 50 compradores simultáneos y 1 pieza', async () => {
  await darDeAltaStock({ variantId, qty: 1 });

  const resultados = await Promise.all(
    Array.from({ length: 50 }, () => api.post('/sales', ticketDeUnaPieza))
  );

  expect(resultados.filter(r => r.status === 201)).toHaveLength(1);
  expect(resultados.filter(r => r.status === 409)).toHaveLength(49);

  expect(await obtenerSaldo(variantId)).toBe('0.000');
  await expect(verificarIntegridad(variantId)).resolves.toEqual({ discrepancias: 0 });
});
```

Pruebas equivalentes para reserva contra venta, dos devoluciones simultáneas del mismo ticket, dos recepciones de la misma orden, y cierre de caja mientras entra una venta. Todas corren contra **PostgreSQL real**, no contra un simulador: un mock no tiene bloqueos de fila y aprobaría código roto.

§08 Operación

EL PRINCIPIO DE ESTA SECCIÓN

Un respaldo que nunca se ha restaurado no es un respaldo: es una esperanza.

CAPA	FRECUENCIA	RETENCIÓN	SIRVE PARA
WAL continuo (PITR)	continuo	30 días	Restaurar a cualquier segundo. Pérdida máxima: segundos
Base completa	diario 03:00	7 días	Restauración rápida
Volcado lógico	diario 04:00	30 días	Portátil, restaurable en otra versión
Semanal / mensual	dom / día 1	12 sem / 24 m	Errores detectados tarde · requisito fiscal
Manual	bajo demanda	30 días	Antes de una importación o migración

Antes de cada operación peligrosa —importación masiva, migración de esquema, actualización mayor— el sistema toma un respaldo automáticamente y lo identifica con esa operación. Deshacer un error de importación deja de ser una emergencia. Todo se cifra con GPG antes de salir del servidor, y la clave privada no vive en el servidor.

La parte que casi nadie hace

VERIFICACIÓN AUTOMÁTICA DE RESTAURACIÓN · CADA DOMINGO

```
1  Levanta un PostgreSQL limpio en un contenedor
2  Restaura el último respaldo completo
3  Aplica el WAL hasta el último punto
4  Comprobaciones de integridad:
     · el conteo de filas coincide con producción
     · SUM(stock_ledger) reconstruye stock_balances EXACTAMENTE
     · las ventas del último mes cuadran con los pagos
     · no hay folios duplicados ni faltantes
5  Reporta ÉXITO o FALLA por correo
6  Destruye el contenedor
```

Si esto falla un domingo, te enteras ese domingo — no el día que de verdad necesites restaurar. El procedimiento manual se ensaya con cronómetro una vez al trimestre; si tarda más que el objetivo declarado, se arregla el procedimiento, no el número.

OBJETIVO	META
Pérdida máxima de datos (RPO)	< 5 minutos
Tiempo máximo de caída (RTO)	< 1 hora
Recuperar una sola tabla	< 15 minutos

Alertas

La **discrepancia entre el libro de movimientos y los saldos** está al máximo nivel de alerta, junto con respaldo fallido y verificación fallida: significa que la premisa central del sistema se rompió y hay que detener la operación para investigar. Por debajo: latencia de búsqueda, cola de webhooks, tasa de errores, disco, y diferencias de caja por encima del umbral.

Presupuesto de rendimiento

OPERACIÓN	OBJETIVO P95
Búsqueda en el POS (índice local)	< 16 ms — un cuadro
Búsqueda en servidor · 100 000 variantes	< 50 ms
Agregar línea al ticket (validación)	< 100 ms
Cerrar una venta completa	< 300 ms
Consulta de stock por API (200 ids)	< 100 ms
Reporte de un mes	< 3 s

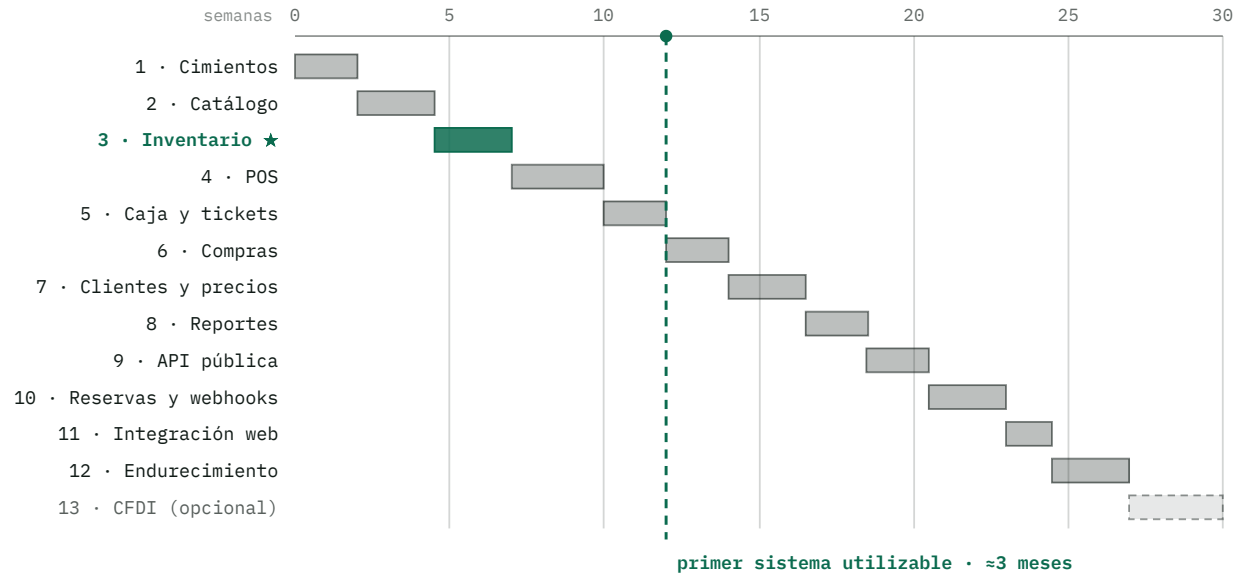
Arranque y migración desde Eleventa

```
1  Semilla del sistema      roles · permisos · métodos de pago · impuestos · canales
2  Configuración inicial    empresa · sucursal · almacén · terminales · folios
3  Usuarios                 administrador + cajeros
4  Importación de catálogo  CSV/Excel con vista previa
5  Carga de inventario      conteo físico inicial → asientos INITIAL_LOAD
6  OPERACIÓN EN PARALELO    2 semanas junto a Eleventa, comparando cortes diarios
7  Corte definitivo         Eleventa queda solo como consulta histórica
```

El paso 6 es innegociable. Migrar un punto de venta en frío es como se pierden dos días de ventas. Dos semanas en paralelo cuestan trabajo, pero convierten un salto al vacío en una transición verificada.

§09 Roadmap

Una fase a la vez. Cada una debe funcionar, estar probada, manejar errores, estar documentada y no romper lo anterior. **No se empieza una fase sin tu aprobación explícita de la anterior.**



Al terminar la Fase 5 ya puedes vender, cobrar, imprimir tickets y controlar inventario — es decir, reemplazar a Eleventa en lo esencial y construir el resto sobre un sistema que ya está en producción real, no en un entorno de pruebas.

Criterios de aceptación que definen cada fase

Cada fase tiene su lista completa; estos son los que de verdad deciden si está terminada o no:

FASE	LA PRUEBA QUE IMPORTA
2 · Catálogo	Escribir `post m` encuentra la faja postquirúrgica talla M · búsqueda < 50 ms con 50 000 variantes
3 · Inventario	50 ventas simultáneas con 1 pieza → 1 éxito, 49 rechazos, saldo 0 · integridad sin discrepancias tras 10 000 movimientos
4 · POS	Venta de 10 artículos en menos de 30 segundos, solo con teclado · el escáner agrega sin ningún clic
5 · Caja	$1 500 pagados con $500 efectivo + $1 000 tarjeta se registran bien · el corte Z cuadra
6 · Compras	Ordenar 100 y recibir 80 deja 20 pendientes y suma 80 al inventario, al costo prorrateado
7 · Clientes	Dos devoluciones simultáneas del mismo ticket no devuelven de más
8 · Reportes	La utilidad usa el costo congelado de cada venta, no el costo actual
9 · API	El costo no aparece en ninguna respuesta pública · quitar un campo hace fallar el CI
10 · Reservas	Enviar el mismo pedido dos veces crea **una** venta · la secuencia completa de tu requisito 22 se comporta como la describiste
12 · Cierre	Restauración completa ensayada con cronómetro dentro de la hora · 100 ventas por minuto sostenidas
13 · CFDI	Con el PAC caído, la tienda sigue vendiendo con normalidad y las facturas se timbran solas al restablecerse

≈27 semanas de trabajo de tiempo completo para el alcance principal, más 3 si se activa la facturación fiscal, con el primer sistema utilizable alrededor del tercer mes.

§10 Decisiones pendientes

Pediste explícitamente que no implementara nada a ciegas y que te avisara si detectaba problemas en los requisitos. Esto es lo que encontré, con una propuesta para cada punto.

Resueltas — el arranque está desbloqueado

D1 ✓ Servidor en la nube (cloud-first)

El POS, la administración y la API se conectan al servidor central. La prioridad es que el inventario esté disponible para la tienda física y para la web **independientemente de que una computadora del local se apague o falle**. La contingencia offline se analiza posteriormente, priorizando siempre la consistencia del inventario.

Topología en §01; los tres niveles de contingencia offline y el riesgo de cada uno, en §07.

D2 ✓ CFDI preparado, no improvisado

Sí se contempla facturación CFDI, pero no en la primera versión funcional. Se diseña ahora la capa de integración con un PAC y se capturan desde la Fase 2 los datos fiscales; el timbrado real llega en la Fase 13. **No se implementa ninguna solución fiscal improvisada.**

Diseño completo en §11 · Facturación.

D3 ✓ Una sucursal hoy, multi-almacén por diseño

Arranca con 1 sucursal y 1 almacén principal, tienda y bodega en el mismo espacio. El modelo es multi-sucursal desde la Fase 1 —es la única forma de no tener que migrar cada movimiento histórico de inventario después—, pero **la interfaz oculta esa dimensión mientras haya una sola ubicación**: sin selector de sucursal, sin campos extra, sin pasos adicionales.

Al dar de alta la segunda ubicación, los selectores aparecen solos y las transferencias se habilitan. Sin migración y sin reescribir historial.

P6 ✓ Política de devoluciones configurable

30 días, producto apto, ticket o venta localizable, reembolso preferente por el método original, opción de crédito a favor, cambios permitidos y autorización superior para efectivo y excepciones. Todo editable desde administración, con cada cambio auditado.

Motor de reglas y matriz de autorizaciones en §12 · Devoluciones.

Problemas ya incorporados al diseño

P1 Precios duplicados en dos modelos

Pediste campos `precio_mayoreo`, `precio_distribuidor` y `precio_especial` dentro del producto *y además* un sistema de listas de precios. Son dos representaciones del mismo dato: garantizado, alguien actualizará el campo del producto y no la lista, y el POS mostrará un precio distinto al de la web.

Propuesta: una sola fuente, `price_lists`. La pantalla del producto sigue mostrando las cuatro casillas de siempre — solo que por debajo escriben en la lista. La experiencia que quieres se conserva íntegra; la posibilidad de contradicción desaparece.

P2 El saldo del cliente como campo editable

Es exactamente el error que estamos corrigiendo en el inventario. Un saldo guardado como número editable acaba siendo un número que nadie puede explicar, y cuando el cliente reclama no hay forma de reconstruir cómo llegó ahí.

Propuesta: cuenta corriente por asientos. El saldo es la suma; cada cargo y cada abono tienen fecha, usuario y documento. El cliente recibe un estado de cuenta imprimible, no una cifra que defender.

P3 «Reservado» y «comprometido» no son dos mecanismos

Implementarlos como contadores independientes duplica la lógica de liberación y hace casi seguro que se desincronicen.

Propuesta: un solo mecanismo con dos estados — `HELD` (con vencimiento) = reservado, y `COMMITTED` (sin vencimiento) = comprometido. Los reportes los muestran por separado, como quieres; el motor los trata igual. «Apartado» es `COMMITTED` con propósito `LAYAWAY`.

P4 Offline y «un solo inventario» están en tensión real

Desarrollado en §07. **Propuesta:** offline por niveles con el riesgo declarado en cada uno. Nivel 0 (lectura) desde la Fase 4, riesgo cero. Nivel 1 (venta en efectivo) en la Fase 12, con colchón de seguridad configurable. Nivel 2 descartado.

P5 Falta el manejo de costos de importación

Importas de Colombia: el costo real de una faja incluye flete internacional, aduana, impuestos de importación y traslado local. Si el sistema registra solo el costo de factura, **todos tus márgenes serán optimistas de forma sistemática**.

Propuesta: ya está en el modelo. Los gastos se prorratean por valor o por peso, y lo que entra al inventario es el costo real puesto en bodega.

ÚNICA PREGUNTA ABIERTA · SE NECESITA ANTES DE LA FASE 6

¿Cómo llega hoy la mercancía y qué gastos adicionales tiene? ¿Compras en pesos mexicanos, dólares o pesos colombianos? Si es en otra moneda, ¿a qué tipo de cambio se registra: el del día de la compra, el del pago, o uno fijo? Esto cambia el modelo de costos y conviene definirlo antes de recibir la primera compra en el sistema.

P7 Códigos de barras duplicados entre proveedores

Los proveedores reutilizan y cambian códigos EAN. Un solo campo en el producto obliga a elegir cuál conservar. **Propuesta:** tabla aparte, varios códigos por variante, unicidad global; ante una colisión el sistema muestra a qué variante pertenece en lugar de un error seco.

P8 Redondeo de efectivo

Un total de $1 499.30 no se puede cobrar exacto en efectivo. **Propuesta:** columna de ajuste por redondeo con regla configurable, aplicada solo cuando el pago es en efectivo. El ajuste aparece en el ticket y en el corte, para que el arqueo cuadre al centavo.

Confirmaciones menores

#	PREGUNTA	VALOR POR DEFECTO PROPUESTO
C1	¿Se permite vender con stock en cero?	No, salvo permiso explícito de gerente
C2	Vida de una reserva web	`20 minutos`
C3	Descuento máximo de un cajero sin autorización	`10 %`
C4	¿El POS pide cliente en cada venta?	No, opcional
C5	¿Cuántas terminales de caja?	Configurable; el diseño soporta N
C6	¿Báscula o venta por pieza?	Solo pieza (el esquema ya admite decimales)
C7	Hora de corte del día operativo	`00:00 hora de México`
C8	¿Apartados con anticipo?	Sí, incluidos como `LAYAWAY`
C9	Idioma y moneda	Español México · MXN
C10	¿Cuántos productos y variantes tienes hoy?	**— necesario para dimensionar**

Lo que este proyecto no va a hacer

Para que quede escrito y no haya malentendidos:

- ✕ Desarrollar, modificar o reemplazar tu página web
- ✕ Construir tienda online, carrito o pasarela de pago
- ✕ Integrar Mercado Libre, WhatsApp, Shopify o WooCommerce en este alcance — la arquitectura queda lista para agregarlos como módulos independientes

- ✕ Contabilidad o nómina
- ✕ Timbrar facturas CFDI, salvo que elijas la opción C en D2
- ✕ Almacenar datos de tarjetas de crédito, nunca

DATOS ÚTILES CUANDO LOS TENGAS A LA MANO

No bloquean nada, pero cuanto antes lleguen, mejor sale la Fase 2: una exportación de tu catálogo actual de Eleventa aunque esté sucio —sirve para diseñar el importador contra datos reales, no contra suposiciones—, una foto de un ticket actual para conservar el formato que tus clientes ya reconocen, el modelo de la impresora térmica y del lector, el volumen aproximado de tickets por día, y el dominio que quieres usar para la API.

§11 Capa de facturación CFDI

DECISIÓN D2

Sí se contempla facturación CFDI, pero **no en la primera versión funcional**. Se diseña ahora la capa de integración y se capturan desde el día uno los datos que hace falta; el timbrado real con un PAC llega después. **No se implementa ninguna solución fiscal improvisada.**

El principio que gobierna el módulo

> `La venta y la factura son dos documentos distintos, con dos tiempos distintos.`

La venta se cierra, el ticket se imprime y el inventario se descuenta **sin esperar jamás al SAT**. La factura es un documento posterior generado a partir de una venta ya cerrada. Tres consecuencias que importan a diario:

- **El cajero nunca espera un timbrado.** Si el PAC está caído o el SAT no responde, la tienda sigue vendiendo con normalidad.
- Un fallo de timbrado **no puede corromper** una venta ni un movimiento de inventario.
- El módulo fiscal se activa, se apaga, se reintenta o cambia de proveedor **sin tocar el POS**.

Esto lo separa de la mayoría de los sistemas del mercado, donde el timbrado va dentro de la transacción de venta y una caída del PAC detiene el mostrador.

Qué se hace ahora y qué después

MOMENTO	ALCANCE	COSTO
Fase 2	Catálogos del SAT y campos fiscales en empresa, cliente, producto y métodos de pago. Validación de RFC por formato	`≈3 días`
Fase 5	El ticket imprime folio e importe con el formato que permite autofacturar después	`incluido`
Fase 7	Las devoluciones nacen preparadas para generar CFDI de Egreso	`incluido`
Fase 13	Timbrado real, factura global, complemento de pago, cancelación, autofacturación	`≈3 semanas`

Por qué pagar esos 3 días ahora. Los datos fiscales son datos de *catálogo*: clave del SAT por producto, régimen fiscal por cliente, código postal fiscal. Capturarlos después significa volver a

tocar producto por producto y cliente por cliente. Capturarlos desde el inicio no cuesta casi nada, porque se llenan mientras se da de alta el catálogo por primera vez.

Para una distribuidora de fajas, prácticamente todo el catálogo cae en una o dos claves de producto del SAT. El alta masiva se resuelve asignando una **clave por defecto a nivel de categoría** que el producto hereda; solo las excepciones se capturan a mano.

EL ERROR QUE RECHAZA MÁS FACTURAS

El `codigo_postal_fiscal` del receptor es obligatorio en CFDI 4.0 y debe coincidir con el domicilio fiscal registrado en el SAT. Si no coincide, el timbrado se rechaza. Por eso se captura y valida **desde el alta del cliente**, no en el momento de facturar, cuando el cliente ya se fue.

La abstracción del PAC

Esta es la pieza central de lo que pediste dejar preparado:

MODULES/FISCAL/PROVIDERS/STAMPING-PROVIDER.INTERFACE.TS

```
export interface FiscalStampingProvider {
  readonly name: string;

  /** Timbra un CFDI. Devuelve UUID, XML sellado y datos del timbre. */
  stamp(payload: CfdiPayload, opts: StampOptions): Promise<StampResult>;

  /** Solicita cancelación. Motivo obligatorio; el 01 exige UUID sustituto. */
  cancel(uuid, motivo: MotivoCancelacion, uuidSustituto?): Promise<CancelResult>;

  /** Consulta el estatus REAL del comprobante ante el SAT. */
  status(uuid): Promise<SatDocumentStatus>;

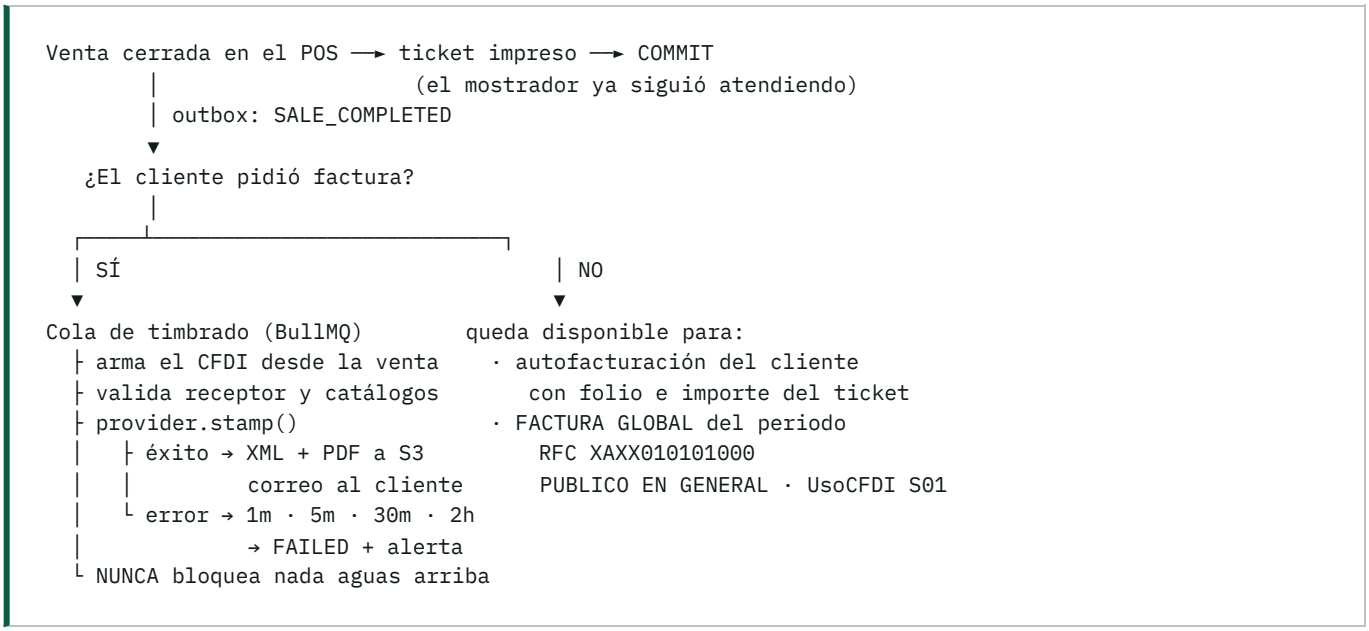
  /** Valida que un RFC exista y esté activo. */
  validateRfc(rfc): Promise<RfcValidation>;

  /** Timbres restantes, para alertar antes de quedarse sin. */
  balance(): Promise<{ remaining: number }>;
}

Implementaciones intercambiables:
  FacturamaProvider · SwSapienProvider · FinkokProvider · MockProvider
```

Cambiar de PAC es cambiar una variable de entorno. Ninguna otra parte del sistema conoce al proveedor. Importa más de lo que parece: los PAC cambian de precio, de límites y de calidad de servicio, y quedar amarrado a uno es una decisión que se paga durante años. El `MockProvider` permite desarrollar y probar todo el flujo, incluidos los casos de error, sin gastar timbres ni usar credenciales reales.

Flujo de emisión



Regla de seguridad: una venta ya incluida en una factura global no puede facturarse individualmente después. El sistema lo bloquea y explica por qué, en vez de generar una duplicidad que el SAT detectaría más tarde.

La **autofacturación** —el cliente factura solo desde un enlace público, con el folio y el importe de su ticket— evita capturar datos fiscales en el mostrador, que es donde se cometen los errores y se hace lento el cobro.

Custodia del sello digital

El CSD es, legalmente, la firma de tu empresa.

RIESGO	CONTROL
Robo del archivo `.key`	Cifrado AES-256-GCM. La clave maestra vive en el gestor de secretos, **nunca en la base ni en el repositorio**
Respaldo comprometido	Los respaldos ya van cifrados con GPG; el CSD queda doblemente cifrado
Uso no autorizado	Timbrar exige permiso `fiscal.stamp`; todo timbrado y cancelación quedan en auditoría
Certificado vencido	Alerta a los 60, 30 y 7 días. Un CSD vencido detiene la facturación por completo

Lo que este módulo deliberadamente no hace

- ✕ **No contabiliza.** No genera pólizas ni balanza. Es facturación, no contabilidad
- ✕ **No presenta declaraciones.** Eso es trabajo de tu contador con su propio software
- ✕ No implementa complementos que no necesitas (Carta Porte, Comercio Exterior, Nómina)
- ✕ **No factura sin CSD real.** No hay «modo simulado» en producción: o está bien configurado, o el módulo está apagado y las ventas se acumulan para la factura global

§12 Política de devoluciones

Una política escrita en el código significa que cada cambio de criterio —una temporada alta, una promoción, un cliente mayorista— es un despliegue. Aquí las reglas viven en configuración, con tipo, validación, valor por defecto e historial.

EL POS CONSULTA ANTES DE QUE EL CAJERO CAPTURE NADA

```
ReturnPolicyService.evaluate(venta, líneas, usuario)
                          |
                          ▼
{ permitida: boolean
  resoluciones_disponibles: [REFUND_SAME_METHOD, STORE_CREDIT, EXCHANGE]
  autorizaciones_requeridas: ['return.refund_cash']
  violaciones: [{ regla, mensaje, subsanable_con_permiso }] }
```

Así el cajero ve **de entrada** qué puede hacer y qué necesita autorización, en lugar de capturar todo y toparse con un error al final.

Configuración inicial

EDITABLE DESDE ADMINISTRACIÓN · CADA CAMBIO QUEDA AUDITADO

```
# Ventana de tiempo
returns.window_days:                    30

# Localización de la venta original
returns.require_original_sale:          true    ← SIEMPRE, no configurable
returns.require_physical_ticket:        false   ← basta con localizar la venta

# Resolución
returns.preferred_refund_strategy:      SAME_METHOD
returns.allow_store_credit:             true
returns.allow_exchange:                 true
returns.allow_partial:                  true
returns.allow_cash_refund_for_card_sale: false   → genera crédito a favor
returns.store_credit_expiry_days:       365
returns.promo_items_policy:             STORE_CREDIT_ONLY

# Autorización
returns.auth.cash_refund:               'return.refund_cash'
returns.auth.amount_threshold:          1000.00
returns.auth.outside_window:            'return.override_window'
returns.auth.damaged_restock:           'inventory.adjust'
```

Una política de devoluciones que se puede cambiar sin dejar rastro es un agujero por donde se va mercancía. Por eso cada modificación registra quién, cuándo, el valor anterior y el nuevo.

Localizar la venta original

Que toda devolución esté ligada a su venta **no es configurable**: sin esa regla, el inventario y la caja dejan de ser explicables. Lo flexible es *cómo* se encuentra esa venta, para cubrir tu criterio de «ticket o venta localizable» sin dejar al cajero atorado cuando el cliente llega sin papel:

```
Escaneo del código del ticket   ← instantáneo, es el camino normal
Folio tecleado                    A-001-000045
Últimos dígitos del folio         "45" busca entre los recientes de la sucursal
Por cliente                       sus compras, de la más reciente hacia atrás
Por fecha e importe               para el caso del ticket perdido
Por producto                      "¿quién compró esta faja en los últimos 30 días?"
```

Reembolso por el método original

Tu regla tiene una consecuencia práctica que conviene dejar escrita, porque cada método se comporta distinto:

MÉTODO ORIGINAL	QUÉ HACE EL SISTEMA
Efectivo	Registra la salida de caja y actualiza el arqueo. Si no hay efectivo suficiente en el cajón, ofrece crédito a favor o exige autorización para un retiro de bóveda
Tarjeta	Registra el pago negativo con la referencia de autorización que captura el cajero. **La devolución física la hace la terminal bancaria, fuera del sistema**
Transferencia	Queda `PENDING` hasta que alguien confirma el traspaso — evita dar por devuelto un dinero que todavía no salió
Crédito de cliente	Revierte el cargo en la cuenta corriente. Un asiento, no un ajuste de saldo
Pago mixto	Reparte proporcionalmente entre los métodos usados, sin devolver por ninguno más de lo que entró por él

POR QUÉ EL SISTEMA NO TOCA LA TERMINAL BANCARIA

Es deliberado. Mantiene el sistema fuera del alcance pesado de PCI y evita el peor escenario posible: un sistema que cree haber devuelto un dinero que el banco nunca movió. El sistema registra y concilia; no opera la terminal.

Autorización escalonada

SITUACIÓN	REQUIERE
Devolución dentro de política, con ticket	**Nadie. El cajero procede**
Devolución en efectivo	Permiso `return.refund_cash`
Monto por encima del umbral	PIN de supervisor
Fuera de los 30 días	Permiso `return.override_window` + motivo escrito
Producto en mal estado que se quiere revender	Permiso `inventory.adjust`
Sin ticket ni venta localizable	**Bloqueado. No hay excepción**

Efectos automáticos, en una sola transacción

```
╔═ TRANSACCIÓN ═══════════════════════════════════════════════╗
║ · Bloquea las líneas de la venta original (FOR UPDATE)      ║
║     → impide que dos terminales devuelvan lo mismo a la vez ║
║ · Valida  SUM(ya devuelto) + nuevo  <=  vendido             ║
║ · Crea la devolución con el PRECIO y el COSTO ORIGINALES    ║
║ · INVENTARIO  GOOD    → almacén de venta, al costo con el que salió ║
║               DAMAGED → almacén de mermas                   ║
║ · CAJA        movimiento de salida, si el reembolso es en efectivo ║
║ · CLIENTE     asiento en cuenta corriente o nota de crédito ║
║ · FISCAL      marca la venta para CFDI de Egreso (Fase 13)  ║
║ · AUDITORÍA   quién devolvió, quién autorizó, motivo        ║
╚═══════════════════════════ COMMIT ═══════════════════════════╝
```

El bloqueo de las líneas originales resuelve un caso real que casi ningún POS contempla: **el mismo ticket presentado en dos cajas al mismo tiempo.** Sin él, se puede devolver más de lo que se vendió.

Devoluciones del canal web

```
Solicitud desde la web  →  estado PENDING
                           ↓  el inventario NO se mueve todavía
La mercancía llega físicamente y se revisa
                           ↓
Recepción en el sistema →  estado COMPLETED
                           ↓  ahora sí: inventario + reembolso
```

El inventario no se incrementa cuando el cliente dice que va a devolver, sino cuando la faja está de vuelta en la bodega y alguien la revisó. Es la diferencia entre un inventario que refleja la realidad y uno que refleja intenciones.

Lo que esta configuración habilita en reportes

- **Tasa de devolución por producto** — señala modelos o tallas con problema de calidad o de tabla de medidas, que en fajas es información de compra directamente accionable.
- Devoluciones por cajero comparadas con el promedio — detección de patrones inusuales.
- Mercancía devuelta en mal estado y su costo acumulado.
- Créditos a favor emitidos, usados y por vencer.
- Devoluciones fuera de política y quién las autorizó.

SIGUIENTE PASO

El diseño está cerrado y las decisiones bloqueantes resueltas. Lo que sigue es aprobar el inicio de la **Fase 1 — Cimientos**: monorepo, esquema completo de 52 tablas, núcleo transversal (transacciones, auditoría, idempotencia, outbox, folios) y autenticación con los seis roles. Sus criterios de aceptación están en §09 · Roadmap.

§13 Fase 1 — Cimientos

CERRADA Y VERIFICADA

155 pruebas en verde — 79 unitarias y 76 de integración contra **PostgreSQL 17.2 real**, no simuladores. Migraciones y seed ejecutados desde una base completamente vacía, `pnpm dev` levantando los cuatro servicios, y los permisos verificados por HTTP contra el API corriendo.

VERIFICACIÓN	RESULTADO
Lint (ESLint + reglas arquitectónicas)	**0 problemas**
Compilación · 6 paquetes	**6/6**
Verificación de tipos · 9 tareas	**0 errores**
Pruebas unitarias	**79/79**
Pruebas de integración · PostgreSQL real	**76/76**
Migraciones desde base vacía	**4 migraciones**
Seed desde cero	**idempotente**
`pnpm dev` levanta los 4 servicios	**API · Panel · POS · BD**
Permisos en backend, por HTTP real	**cajero → 403**

La prueba que define el sistema

EJECUTADA CONTRA POSTGRESQL 17.2, NO CONTRA UN SIMULADOR

```
50 compradores simultáneos · 1 pieza en existencia
  → 1 venta exitosa
  → 49 rechazos con HTTP 409
  → saldo final: 0 (nunca negativo)
  → 1 solo asiento en el kardex (sin huérfanos de transacciones revertidas)

Y además:
  · 20 compradores con 5 piezas     → venden exactamente 5
  · 60 operaciones concurrentes     → SUM(kardex) reconstruye el saldo EXACTO
  · la web reserva la última pieza  → el mostrador NO puede venderla
  · UPDATE que ignora la validación → rechazado por la restricción CHECK
  · 30 folios simultáneos           → únicos y SIN HUECOS (1…30)
  · misma clave idempotente ×20      → una sola gana
  · dos productos distintos         → se venden en paralelo sin estorbarse
```

Permisos: verificado contra el API corriendo

```
Cajero (16 permisos) llamando DIRECTO al API, sin pasar por la interfaz:

  GET   /users                → 403   falta: user.view
  GET   /users/permissions    → 403   falta: role.manage
  POST  /users                → 403
  PATCH /users/:id/roles      → 403   (intento de autoasignarse ADMIN)
        roles después         → CASHIER   ← no cambió nada

Administrador (76 permisos):
  GET   /users                → 200
  GET   /users/permissions    → 200
```

Levantar el proyecto en una máquina limpia

DOS COMANDOS

```
pnpm install
pnpm setup        ← idempotente: se puede repetir sin romper nada

  [1/5] Verifica Node, pnpm y Docker
  [2/5] Crea el .env con secretos ÚNICOS generados al vuelo
  [3/5] Levanta PostgreSQL y espera a que responda
  [4/5] Genera el cliente Prisma y aplica las migraciones
  [5/5] Siembra permisos, roles, catálogos, empresa, sucursal y administrador

pnpm dev          ← arranca API + panel + POS

  API             http://localhost:3000/api/v1
  Documentación   http://localhost:3000/docs
  Administración  http://localhost:5173
  Punto de venta  http://localhost:5174
```

REQUISITO	VERSIÓN	COMPROBAR
Node.js	22 o superior	node -v
pnpm	10 o superior	npm install -g pnpm
Docker Desktop	reciente	docker --version
Git	cualquiera	git --version

Estructura entregada

```
apps/
  api/         NestJS 11 · el Sistema Central
  admin/       React 19 + Vite · panel administrativo
  pos/         React 19 + Vite · punto de venta
packages/
  shared-types/  permisos, roles, errores, eventos — compartidos API ↔ frontends
  domain-rules/  dinero, costeo promedio, redondeo, prorrateo, folios
  config/        tsconfig compartido
infra/postgres/init/   creación del rol restringido de aplicación
scripts/               setup.mjs · wait-for-db.mjs
```

Base de datos

76

Tablas

71 del modelo operativo más 5 catálogos del SAT. Medidas contra la base, no estimadas.

240

Índices

Compuestos, parciales, GIN trigram y de texto completo.

123

Llaves foráneas

Integridad referencial declarada en la base, no en el código.

24

Restricciones CHECK

Las tres del inventario, más reservas, pagos, devoluciones, monedas y zonas horarias.

EL CONTEO: 52 → 76

Un borrador temprano decía «52 tablas». **Ese número era simplemente incorrecto**: un conteo hecho a ojo al redactar el resumen, que ni siquiera cuadraba con el desglose por dominio del mismo documento (sumaba 72). No hubo alcance añadido a escondidas; hubo un resumen mal contado.

```
71  modelo operativo
 5  catálogos del SAT      (decisión D2 — preparación fiscal)
──
76  TOTAL
```

De las 71 operativas, 2 se añadieron en esta fase por tu punto 10: `landed_costs` y `exchange_rates`. Sin D2 y sin el punto 10, el modelo son 69 tablas. El desglose por dominio, con la justificación de cada tabla puente, de detalle y de historial, está en el modelo de datos.

DOS MIGRACIONES

MIGRACIÓN	CONTENIDO
…000_init	Las tablas base, generadas desde el esquema Prisma
…001_hardening	Todo lo que Prisma no sabe modelar: extensiones, los CHECK, la columna generada `qty_available`, índices únicos parciales, GIN trigram y FTS, el trigger de normalización de búsqueda, y los `REVOKE` del kardex y la auditoría
…002_timezone	Zona horaria obligatoria en la empresa, heredable en la sucursal
…003_landed_costs	Costos de importación desglosados y tipos de cambio históricos

La decisión más importante de la fase

DOS ROLES DE BASE DE DATOS

```
pv_owner   dueño del esquema · ejecuta migraciones y seed
pv_app     rol de la APLICACIÓN · restringido

REVOKE UPDATE, DELETE ON "stock_ledger" FROM pv_app;
REVOKE UPDATE, DELETE ON "audit_logs"   FROM pv_app;
```

Esto convierte «el kardex es inmutable» y «la auditoría no se puede editar» de una convención del equipo en una **regla de PostgreSQL**. Ni un bug ni alguien con la credencial de la aplicación pueden reescribir historial. Está verificado con pruebas de integración que intentan el `UPDATE` y esperan que la base lo rechace.

Núcleo transversal

PIEZA	QUÉ RESUELVE
`PrismaService`	Transacciones con contexto ambiental (`AsyncLocalStorage`), traducción de `lock_timeout`, y `lockStockBalances()` con **orden determinista** para prevenir deadlocks
`AuditService`	Registro inmutable con diff campo a campo; redacta contraseñas, PIN y tokens
`OutboxService`	Publica eventos **en la misma transacción** que el dato
`OutboxRelay`	Barrido con `FOR UPDATE SKIP LOCKED`: varias instancias comparten la cola sin duplicar
`IdempotencyService`	Reclamo atómico de clave, hash estable del cuerpo, reintento tras fallo
`SequenceService`	Folios `A-001-000123` con `UPDATE … RETURNING`: **sin huecos**
`CryptoService`	AES-256-GCM para secretos recuperables; comparación en tiempo constante
`RequestContext`	`request_id` correlacionado en logs, auditoría y kardex

Autenticación

- **argon2id** (64 MB, t=3, p=4) para contraseñas y PIN — no bcrypt.
- **PIN de POS:** la terminal se identifica por dispositivo, el cajero por PIN.
- **Autorización de supervisor:** verifica que quien autoriza tenga de verdad el permiso, y lo deja registrado en auditoría.
- **Refresh rotatorio con detección de reutilización:** un token ya consumido revoca toda la familia de sesiones — la señal clásica de robo.
- **Verificación señuelo** para que el tiempo de respuesta no revele qué correos existen.
- Token de acceso en memoria; refresco en cookie `httpOnly` + `SameSite=Strict`.

Los 6 roles, con permisos aplicados en el backend

ROL	PERMISOS	ALCANCE
Administrador	76	Todo
Gerente	70	Todo salvo usuarios, roles, respaldos, API y configuración fiscal
Cajero	16	Vender, cobrar, su caja, devolver con autorización
Almacén	17	Inventario, conteos, transferencias, recepción
Ventas	9	Consulta y clientes; no cobra
Solo lectura	14	Consulta y reportes sin utilidad ni costos

76 permisos en 8 grupos, aplicados por un guard **global**: todo endpoint exige sesión salvo que declare `@Public()`. Los permisos se resuelven **contra la base en cada petición**, no se leen del token: retirar un permiso surte efecto de inmediato, no cuando caduque la sesión.

Pruebas

79 PRUEBAS UNITARIAS · TODAS EN VERDE · SIN BASE DE DATOS

```
@pv/domain-rules   29 pruebas
  money.spec.ts    20  aritmética exacta, costeo promedio, redondeo de
                       efectivo, prorrateo de importación, impuestos
  folio.spec.ts     9  formato y lectura de folios

@pv/api            50 pruebas
  ids.spec.ts       4  UUID v7: unicidad y orden temporal
  core.spec.ts     28  validación de entorno, cifrado, idempotencia,
                       auditoría, errores de dominio
  permissions       18  el guard y la definición de los 6 roles
```

Casos que verifican requisitos concretos del diseño:

- El costeo promedio reproduce **exactamente** el kardex documentado: $400 → $437.50 → $430.61.
- El redondeo de efectivo resuelve $1 499.30 → $1 499.50 con ajuste de $0.20.
- El prorrateo de flete **cuadra al centavo**: el residuo del redondeo no se pierde.
- La auditoría **nunca** incluye contraseñas, PIN ni tokens.
- El API **no arranca** con secretos de ejemplo en producción.

- El Cajero no puede ver costos, ajustar inventario ni cancelar ventas.

PENDIENTE DE EJECUTAR

Hay **18 pruebas de integración escritas** contra PostgreSQL real: restricciones CHECK, append-only por permisos, unicidad parcial, normalización de búsqueda, idempotencia, folios sin huecos, y el bloqueo de fila serializando dos transacciones sobre el mismo saldo. **No se han ejecutado porque esta máquina no tiene Docker.** Se corren con `pnpm db:up && pnpm test:integration` en cuanto se instale.

Defecto encontrado y corregido durante la fase

El valor de ejemplo de `ENCRYPTION_KEY` decodificaba a **36 bytes en vez de 32**, así que el API se negaba a arrancar recién clonado el repositorio. Lo detectó su propia validación de entorno al probar el arranque, y quedó corregido en los cuatro archivos de entorno.

La validación funcionó como se diseñó: falló al arrancar, con un mensaje que decía exactamente qué estaba mal.

Deuda técnica declarada

#	ASUNTO	PROPUESTA
D-1	**Particionado del kardex**	**Aplazado a la Fase 3.** Prisma no modela tablas particionadas y con cero movimientos no acelera nada. Es una decisión de esquema: **no la tomé por mi cuenta**, necesito tu visto bueno
D-2	**`prisma migrate dev` no se usa**	Propondría borrar los objetos del endurecimiento. El flujo es `db:migrate:new` → revisar el SQL → `db:deploy`
D-3	**`qty_available` fuera del cliente**	Es columna generada por PostgreSQL. Se lee con SQL crudo, que es lo que el inventario ya usa para bloquear
D-4	**Redis declarado, no activo**	Bajo perfil `queue`, no arranca por defecto. Se activa en la Fase 10 con BullMQ
D-5	**Frontends mínimos**	Login y permisos efectivos. El sistema de diseño entra en la Fase 2, con pantallas reales que diseñar
D-6	**MFA en el esquema, sin flujo**	Los campos existen; el flujo TOTP va en la Fase 12

Criterios de aceptación

CRITERIO	ESTADO
Monorepo estructurado	**6 paquetes · Turborepo · pnpm**
Backend, frontend y paquetes compartidos	**Compilan y arrancan**
PostgreSQL vía Docker	**Solo lo necesario**
Esquema completo y migraciones	**74 tablas · 2 migraciones**
Relaciones, índices, restricciones, llaves	**146 · 122 · 15**
Transacciones y concurrencia en el núcleo	**Contexto ambiental + bloqueo determinista**
Sistema de auditoría	**Con diff, redacción e inmutabilidad**
Idempotencia	**Reclamo atómico, hash estable**
Patrón Outbox	**Transaccional + relay**
Sistema de folios	**Por sucursal, sin huecos**
Autenticación segura	**argon2id + rotación con detección**
6 roles aplicados **en backend**	**Guard global + 18 pruebas**
Entornos separados	**4 archivos, validados al arrancar**
Seed inicial	**Idempotente**
Scripts de instalación y desarrollo	`pnpm setup` **+ 14 comandos**
Documentación para máquina limpia	**README + docs/13**
Pruebas de componentes críticos	**79 en verde**
`pnpm dev` levanta todo	**Falta ejecutarlo con Docker instalado**

SIGUIENTE PASO

Instalar Docker Desktop y ejecutar `pnpm setup && pnpm dev` para verificar el arranque completo y correr las 18 pruebas de integración. Después, aprobar el inicio de la **Fase 2 — Catálogo**: productos, variantes, códigos de barras, el motor de búsqueda y la importación desde Excel.

Antes de la Fase 2 conviene responder **C10**: cuántos productos y variantes tienes hoy, para dimensionar el importador.

Documento de arquitectura e implementación · versión 3 · 5 de septiembre de 2026 · decisiones D1, D2, D3 y P6 resueltas · Fase 1 completada. Para guardarlo en PDF: **Ctrl+P** → **Guardar como PDF** en computadora, o **Compartir** → **Imprimir** → **Guardar como PDF** en el celular. La especificación completa vive en el repositorio del proyecto:

`README.md`
`docs/00-RESUMEN-EJECUTIVO.md` `docs/01-arquitectura.md` `docs/02-modelo-datos.md`
`docs/03-inventario.md` `docs/04-flujos-operativos.md` `docs/05-api.md`
`docs/06-seguridad.md` `docs/07-concurrencia.md` `docs/08-operacion.md`
`docs/09-roadmap.md` `docs/10-decisiones-pendientes.md`
`docs/11-facturacion-cfdi.md` `docs/12-politica-devoluciones.md`
`docs/13-fase-1-cimientos.md`